
REGBOT

Balance Assignment

Group 19

Course 34721: Linear Control Design

AUTHORS

Maja Allermann Johansen - s235193

Nick Gholami - s235175

Frederik Toft Johansen - s235181

Anton Werner Løhndorf - s235186

May 17, 2026

Contents

1	Introduction	1
2	Implementation of a Wheel Speed Controller	1
3	Designing a Balance Controller	2
3.1	Post-integrator	2
3.2	Tilt controller	2
4	Velocity Controller Implementation	3
4.1	Pre-filter and Undershoot Mitigation	4
5	Position Controller Design	4
5.1	Discussion	5
5.2	Conclusion	5
A	Appendix	6
A.1	Voltage-to-velocity transfer function Week 5	6
A.2	Transfer Function - Base system	6
A.3	Transfer Function - Base system and tilt controller	6
A.4	Transfer Function - Base system, balance controller and velocity controller	6
A.5	REGBOT square mission	7
A.6	Mission: Wheel Speed Controller	7
A.7	Mission: Balance controller	7
A.8	Mission: Velocity controller	7
A.9	Mission: Position controller	7
A.10	Simulink Model	9

1 Introduction

The purpose of this assignment is to design and implement a control system for the REGBOT such that it is able to balance itself while performing motion tasks.

2 Implementation of a Wheel Speed Controller

The wheel speed controller forms the inner control loop of the REGBOT balance system. Its purpose is to regulate the wheel speed of both motors accurately, which is essential for maintaining balance. The controller is implemented as a PI-controller with a feed forward constant and a low pass filter to reduce noise. The voltage-to-velocity transfer function identified in week 5 is used as the plant (see appendix A.1). The open-loop transfer function with the PI-controller is given by:

$$G_{wv,ol} = K_{P,wv} \frac{\tau_{i,wv}s + 1}{\tau_{i,wv}s} G_{plant} \quad (1)$$

The PI-controller is designed with the PI-factor, $N_i = 3$, and the phase margin, $\gamma_M = 90^\circ$. The phase at the new crossover frequency is then obtained from the phase balance equation with the phase contribution from the PI-part:

$$\angle G_{plant}(j\omega_c) = -180^\circ + \gamma_M - (-\arctan(\frac{1}{N_i})) = -180^\circ + 90^\circ + 18.4^\circ = -71.6^\circ \quad (2)$$

The new crossover frequency is obtained at $\angle G_{plant}(j\omega_c) = -71.6^\circ$ at the Bode-plot, and $\omega_c = 26 \text{ rad/s}$. The PI time constant and the proportional gain are determined:

$$\tau_{i,wv} = \frac{N_i}{\omega_c} = 0.116, \quad K_{P,wv} = \frac{1}{|C_{PI}(j\omega_c)G_{plant}(j\omega_c)|} = 9.02 \quad (3)$$

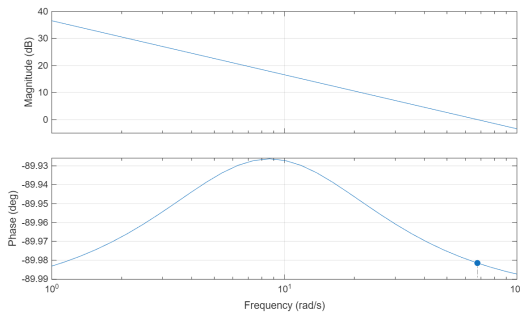


Figure 1: Open-loop Bode-plot, with $\omega_c = 67.6 \text{ rad/s}$ and $\gamma_M = 90^\circ$

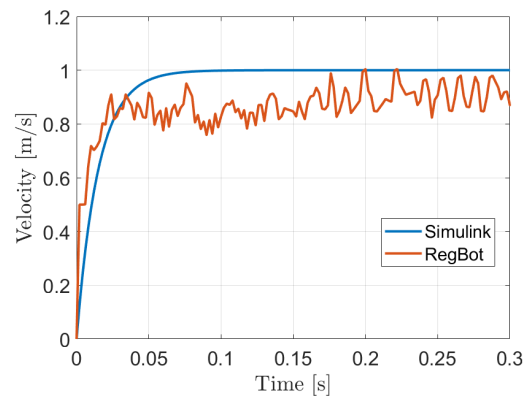


Figure 2: Step Response comparison of the REGBOT data and the Simulink data.

$K_{P,wv}$ was tuned to obtain a faster settling time, and the final tuning resulted in: $K_{P,wv} = 23.5$. An open-loop Bode-plot, of the transfer function from motor voltage to velocity with PI-controller is shown in figure 1. Figure 2 shows a comparison of the simulation step response and the actual REGBOT step response. The mission used can be found in appendix A.6.

3 Designing a Balance Controller

The purpose of the balance controller is to stabilize the unstable dynamics of the REGBOT. A linearisation is made from the voltage reference to the tilt angle reference in Simulink using MATLAB to obtain the transfer function G_{tv} (A.2).

3.1 Post-integrator

The transfer function, G_{tv} , is unstable and therefore another PI-block is added to the system to stabilize it. The transfer function with the post-integrator, $C_{PI,post}(s)$ is given by:

$$G_{tv,post}(s) = -C_{PI,post}G_{tv}(s) \quad (4)$$

The post-integrator time constant is given by:

$$\tau_{i,post} = \frac{1}{\omega_{i,post}} \quad (5)$$

Where $\omega_{i,post}$ is the frequency at the magnitude peak from the $G_{tv}(s)$ Bode-plot. $\omega_{i,post} = 8.40$ rad/s. The new post-integrator transfer function, $G_{tv,post}(s)$, was then added to the Simulink system with a gain block $k = -1$.

3.2 Tilt controller

Now that the system is stable, a PI-Lead controller is implemented for the tilt controller. This controller was chosen to ensure that the REGBOT reaches a precise balance state at the desired point by eliminating steady-state errors, and the Lead-part ensures that the system remains stable and controlled without aggressive overshoot. The parameters γ_M , N_i and α are selected:

$$\gamma_M = 60^\circ \quad N_i = 3 \quad \alpha = 0.015 \quad (6)$$

Hereafter, the phase shift from the I-part, ϕ_i , and the Lead-part, ϕ_m , are calculated to establish the phase balance equation:

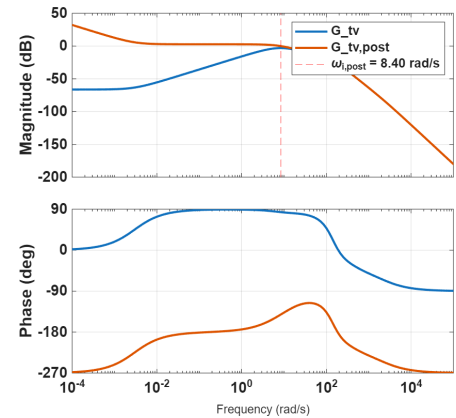


Figure 3: Open loop Bode-plot of G_{tv} and $G_{tv,post}$ with the G_{tv} peak magnitude at $\omega_{i,post}$

$$\begin{aligned}\angle G(j\omega_{c,new}) &= \gamma_M - \left(-\arctan\left(\frac{1}{N_i}\right) \right) - 180^\circ - \arcsin\left(\frac{\alpha-1}{\alpha+1}\right) \\ \Rightarrow \angle G(j\omega_{c,new}) &= 60^\circ + 18.43^\circ - 180^\circ - 76.04^\circ = -177.60^\circ\end{aligned}\quad (7)$$

The new crossover frequency, is identified: $\omega_{c,new} = 147.60$ rad/sec and the initial controller parameters are calculated:

$$\tau_{d,tilt} = \frac{1}{\omega_c \sqrt{\alpha}} = 0.0553, \quad \tau_{i,tilt} = \frac{N_i}{\omega_c} = 0.020, \quad K_{P,tilt} = \frac{1}{|C_{PI}(j\omega_c)C_{Lead}(j\omega_c)G(j\omega_c)|} = 2.406 \quad (8)$$

To achieve an optimal balance between response speed, stability, and disturbance rejection, the parameters were fine-tuned on the robot:

$$\tau_{d,tilt} = 0.05, \quad \tau_{i,tilt} = 1.0, \quad K_{P,tilt} = 2 \quad (9)$$

The performance of the system was evaluated by analyzing the step response, as shown in figure 5. Here the yellow graph shows the step response from the REGBOT, mission at A.7.

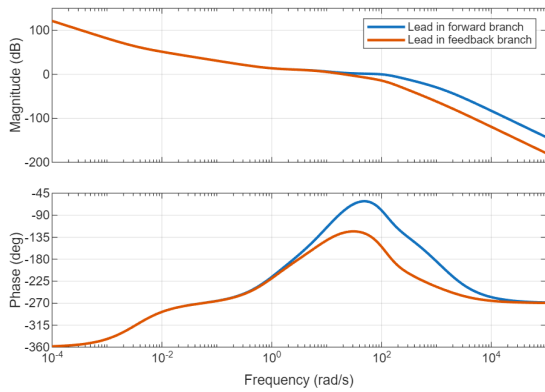


Figure 4: Comparison Bode-plot of the lead in forward branch and backward branch

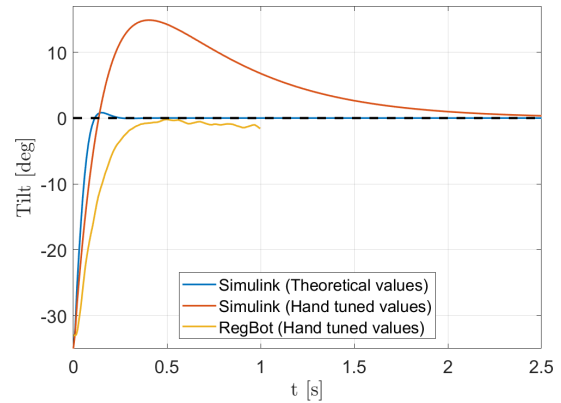


Figure 5: Closed-loop step response with the tilt controller. Mission at A.7.

4 Velocity Controller Implementation

The system was then re-linearized to yield the velocity transfer function $G_{vel}(s)$ (see Appendix A.3). Initially, a PI-Lead compensator was designed in simulation to maximize phase margin and injection damping. However, when implementing the theoretical values an increase in oscillations and motor jitter were seen caused by the lead-part. Therefore through hand tuning the Lead-element was removed, leaving a robust PI-structure:

$$C_{vel}(s) = K_{P,vel} \left(1 + \frac{1}{\tau_i s} \right) \quad (10)$$

Experimental tuning in Simulink yielded $K_{P,vel} = 0.4$ and $\tau_i = 1$ s, which successfully eliminated steady-state error without inducing low-frequency stuttering.

4.1 Pre-filter and Undershoot Mitigation

To handle the characteristic non-minimum phase undershoot of the REGBOT, a first-order pre-filter smooths the reference step to protect the PI controller from aggressive changes in acceleration.

$$G_f(s) = \frac{1}{\tau_f s + 1}, \quad \text{with } \tau_f = 1.5 \text{ s} \quad (11)$$

Because the REGBOT no longer has Lead damping, a high $\tau_f = 1.5$ s is necessary to ramp the velocity set-point exponentially, allowing the tilt loop enough time to adjust the pitch angle smoothly. The combined system characteristics are shown in figure ??.

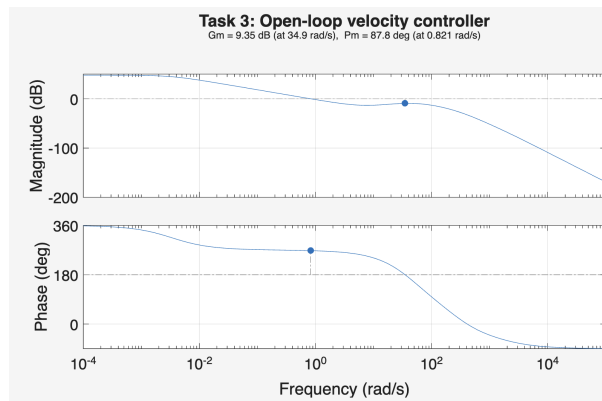


Figure 6: Open-loop Bode-plot for velocity controller, with $\omega_c = 0.82$ rad/s and $\gamma_M = 87.9^\circ$

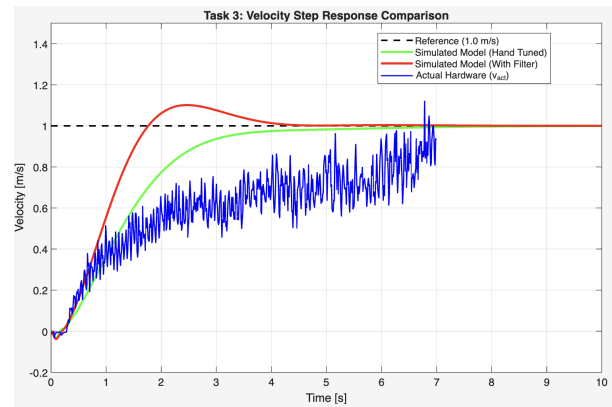


Figure 7: Closed-loop step response comparison. Mission: A.8.

Figure 7 evaluates the velocity step response. The simulated model (red graph) is built on a PI-Lead design with a pre-filter. On the REGBOT, the PI-Lead design failed as its Lead-element amplified the already encountered disturbances, causing huge vibrations and aggressive accelerations. Therefore, the Lead-element was removed from the REGBOT, leaving a robust PI-only design (blue graph).

5 Position Controller Design

The objective of the position controller is to move the REGBOT to a specific coordinate while maintaining stability. After implementing the velocity controller, another linearization was made, $G_1(s)$, shown in Appendix A.4. A P-controller was chosen for the position controller design, because the inner control loops already provide stability, making a simple outer-loop controller sufficient. The initial gain was found using the closed-loop Ziegler-Nichols method,

where the marginal gain was found to be: $K_{P,m} = 1.6$. Using the Ziegler-Nichols rule (Table 3.2 in the book) for a P-controller gives: $K_P = 0.5 \cdot K_{P,m} = 0.8$. However, this resulted in a response that was too aggressive on the REGBOT. Therefore, the gain was reduced through hand tuning. The final controller parameter was selected as: $K_P = 0.35$.

The final controller was evaluated both in Simulink and on the physical REGBOT by a square mission, see 10. Figure 8 shows the open-loop Bode-plot for the position controller, while figure 9 shows the comparison between the simulated and experimental response based on the mission in appendix A.9.

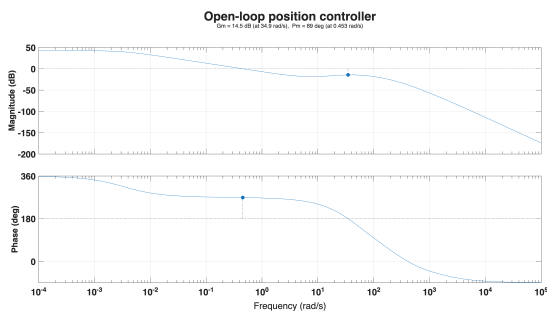


Figure 8: Open-loop Bode-plot of the position controller, with $\omega_c = 0.45$ rad/s and $\gamma_M = 89^\circ$.

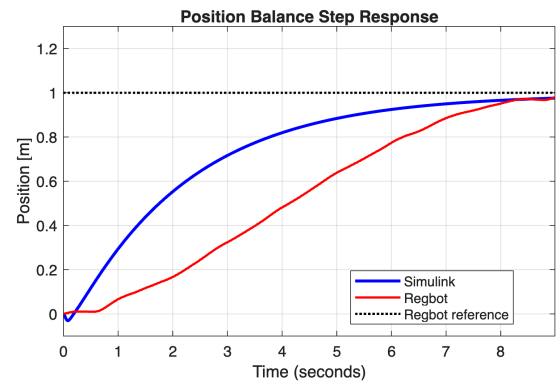


Figure 9: Comparison between simulated and experimental response.

5.1 Discussion

The experimental responses generally followed the same trends as the simulated responses, indicating that the selected controller structure was suitable for the REGBOT system. However, several parameters had to be adjusted manually before satisfactory behaviour was achieved on the physical robot. The deviations between simulation and experiment are likely caused by effects such as friction, delays, sensor noise and modelling inaccuracies, which are not fully represented in the linearized model. This demonstrated that practical tuning was necessary in order to achieve reliable behaviour on the REGBOT.

5.2 Conclusion

In this assignment, controllers for wheel velocity, balance, velocity and position regulation were designed and implemented for the REGBOT using frequency-domain methods and Simulink linearization. The final system was able to balance while performing both motion and positioning tasks. Overall, the controller implementation fulfilled the main objectives of the assignment and demonstrated stable behaviour during experimental testing. A video of the REGBOT running all missions can be seen following the link at the bottom of this page ¹. and the Simulink model is found at A.10.

¹<https://youtu.be/MEZGe73m3MY?feature=shared>

A Appendix

A.1 Voltage-to-velocity transfer function Week 5

$$G_{plant}(s) = \frac{2.8784}{s + 8.6429} \quad (12)$$

A.2 Transfer Function - Base system

Linearization from 'vel_ref' to 'tilt_angle'

$$G_{tv}(s) = \frac{-5.461 \times 10^5 s^4 - 7.145 \times 10^8 s^3 - 1.262 \times 10^{11} s^2 - 1.036 \times 10^{12} s - 3.463 \times 10^9}{s^7 + 2180 s^6 + 1.145 \times 10^6 s^5 + 1.686 \times 10^8 s^4 + 1.231 \times 10^{10} s^3 + 8.404 \times 10^{10} s^2 - 6.703 \times 10^{11} s - 4.955 \times 10^{12}} \quad (13)$$

A.3 Transfer Function - Base system and tilt controller

Linearization from 'theta_ref' to 'tilt_angle'

$$G(s) = \frac{-2.12 \times 10^7 s^6 - 2.158 \times 10^{10} s^5 - 3.828 \times 10^{11} s^4 - 7.274 \times 10^{11} s^3 + 1.929 \times 10^{13} s^2 + 1.029 \times 10^{14} s + 8.328 \times 10^{13}}{s^9 + 2180 s^8 + 1.199 \times 10^6 s^7 + 2.311 \times 10^8 s^6 + 1.636 \times 10^{10} s^5 + 4.821 \times 10^{11} s^4 + 5.448 \times 10^{12} s^3 + 2.248 \times 10^{13} s^2 + 1.748 \times 10^{13} s + 5.821 \times 10^{10}} \quad (14)$$

A.4 Transfer Function - Base system, balance controller and velocity controller

Linearization from 'vel_ref_task3' to 'wheel_vel_filter'

$$G_1(s) = \frac{-5.723 \times 10^6 s^6 - 5.826 \times 10^9 s^5 - 1.033 \times 10^{11} s^4 - 1.964 \times 10^{11} s^3 + 5.208 \times 10^{12} s^2 + 2.779 \times 10^{13} s + 2.249 \times 10^{13}}{s^9 + 2180 s^8 + 1.199 \times 10^6 s^7 + 2.311 \times 10^8 s^6 + 1.636 \times 10^{10} s^5 + 4.821 \times 10^{11} s^4 + 5.448 \times 10^{12} s^3 + 2.248 \times 10^{13} s^2 + 1.748 \times 10^{13} s + 5.821 \times 10^{10}} \quad (15)$$

A.5 REGBOT square mission

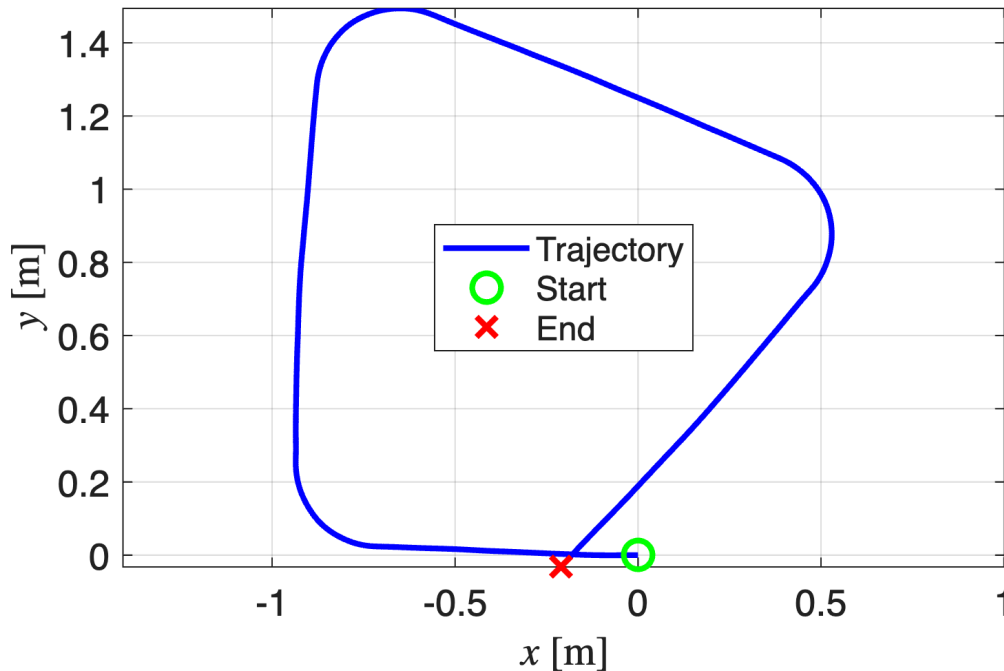


Figure 10: Plot showing the motion of the REGBOT doing the square mission, mentioned under "Position controller design"

A.6 Mission: Wheel Speed Controller

```
vel=0 , log=2 : time=0.5
vel=0 : time=1
log=0
```

A.7 Mission: Balance controller

```
bal=0 , log=2 : time=0.5
bal=1 : time=1
log=0
```

A.8 Mission: Velocity controller

```
bal=1 , vel=0 , log=10 : time=3
bal=1 , vel=1 : time=7
log=0
```

A.9 Mission: Position controller

```
bal=1 , vel=0 , log=10 : time=3
```

```
bal=1 , vel=1 , topos=1 : time=10  
log=0
```

A.10 Simulink Model

